

# COMP3320/6464 Week 7, Lecture 2/3

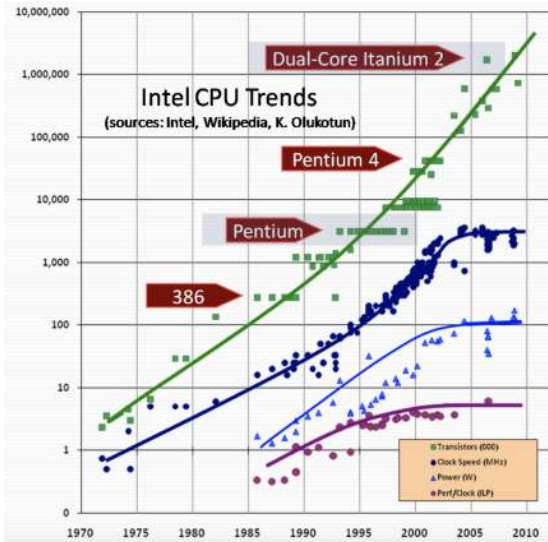
Rhys Hawkins

Semester 2, 2024

# References

- The end of the road for general purpose processors & the future of computing. John Hennessy. <https://www.hanahaus.com/blog-1/2019/1/3/newport-beach-a-fascinating-location-mpsfz-rwrbk>
- The Free Lunch Is Over. A Fundamental Turn Toward Concurrency in Software. Herb Sutter.  
<http://www.gotw.ca/publications/concurrency-ddj.htm>
- Chapter 12 from Computer Systems A Programmer's Perspective, Third Edition, Randal E. Bryant and David R. O'Hallaron, Pearson Education Heg USA, ISBN 9781292101767.
- Programming with POSIX Threads, David R. Butenhof, Addison-Wesley Professional, ISBN-13 : 978-0201633924.

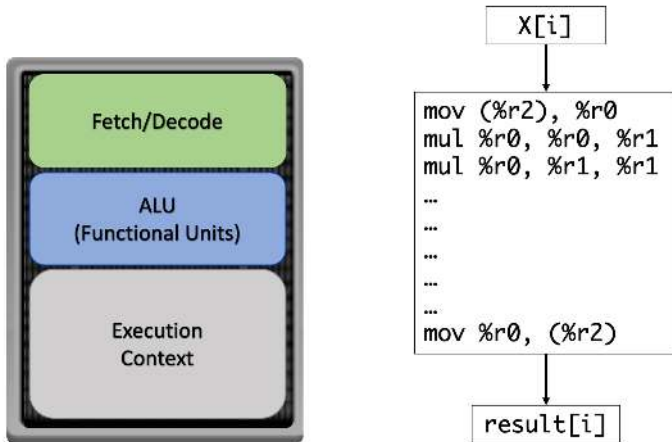
# The end of the uniprocessor era



- Clock rate capped (< 4 GHz) by power constraints
- ILP (superscalarity and pipelining) saturated
- Power saturated (limitations in heat removal)
- The free lunch is over

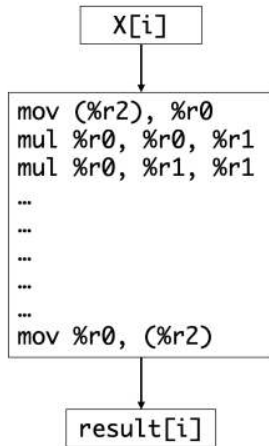
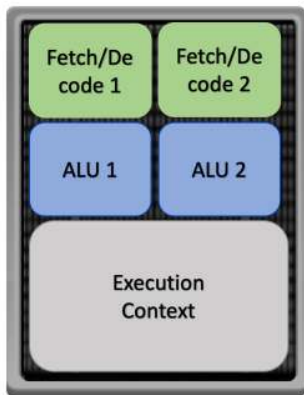
# Limitations of ILP

## Basic serial processor workflow



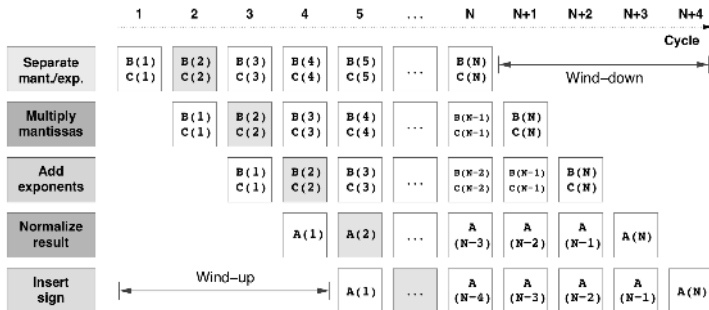
# Limitations of ILP

Superscalarity: fetch/decode and execute multiple instructions per clock (if possible) - needs out-of-order execution



# Limitations of ILP

- Pipeline parallelism: instructions can be broken down into stages that can run concurrently (e.g. fetch-decode-execute stages of single instruction)
- Modern processors have pipelines with 15+ stages (Prescott reached 31!)



**Figure 1.5:** Timeline for a simplified floating-point multiplication pipeline that executes  $A(:) = B(:) * C(:)$ . One result is generated on each cycle after a four-cycle wind-up phase.

# Limitations of ILP

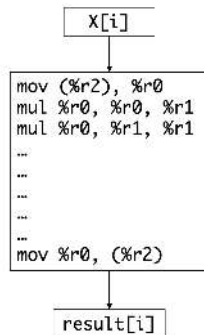
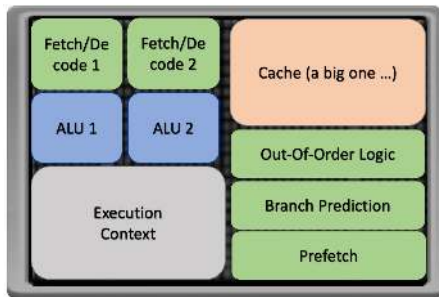
- 4 instructions/clock  $\times$  15-deep pipeline  $\rightarrow$  60 instructions in flight
- Branches and memory aliasing are the key limit:
  - wasteful to start a pipeline without knowing the outcome of branch
  - cannot reorder instruction with aliasing
- Speculation was introduced to allow this
  - Predict branches and predict matching memory addresses
  - we operate as if our guesses are right
  - if the prediction is accurate ... great!
  - if it not, undo the work and start again
- How good must branch prediction be?

# Limitations of ILP

- 4 instructions/clock  $\times$  15-deep pipeline  $\rightarrow$  60 instructions in flight
- Branches and memory aliasing are the key limit:
  - wasteful to start a pipeline without knowing the outcome of branch
  - cannot reorder instruction with aliasing
- Speculation was introduced to allow this
  - Predict branches and predict matching memory addresses
  - we operate as if our guesses are right
  - if the prediction is accurate ... great!
  - if it not, undo the work and start again
- How good must branch prediction be?
  - 15-deep pipeline: 4 branches 94% correct  $\rightarrow$  98.7% accurate
  - 60 instructions in flight: 15 branches 90% correct  $\rightarrow$  99% accurate
  - Intel i7 over 100 instructions in flight
  - The answer: very, very good!

# Limitations of ILP

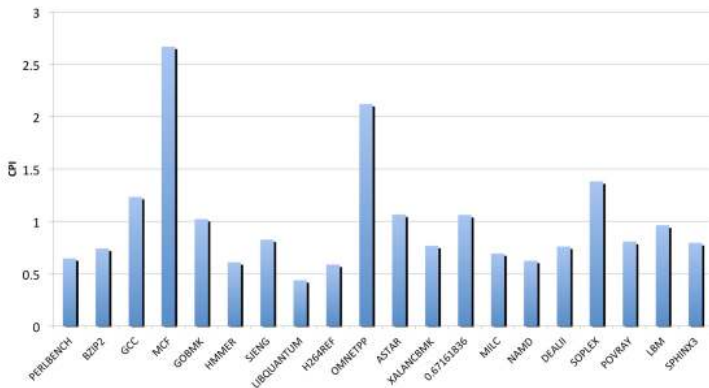
Large number of transistors used for logic that help single instruction stream run faster.



This has diminishing returns.

# Limitations of ILP

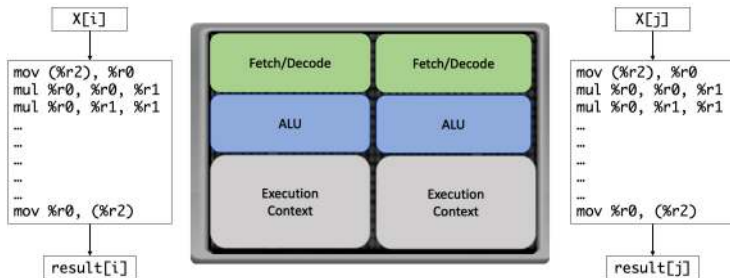
Actual clock cycles per instruction (CPI) on Intel i7. Theoretical is 0.25.



A great deal of performance issues related to not reaching the theoretical minimum CPI are caused by branch prediction failures and having to redo some work (and unwinding/stalling pipelines).

# The multicore era

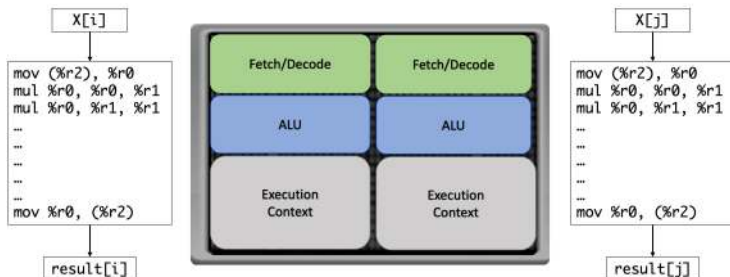
Rather than use transistors to increase the sophistication of single instruction stream, use additional transistors to add more cores.



- 2 cores, each one slightly slower (e.g. 0.75) than my original processor.
- Potential speedup 1.5x.
- Problem: what happens if we run our program on this new processor?

# The multicore era

Rather than use transistors to increase the sophistication of single instruction stream, use additional transistors to add more cores.

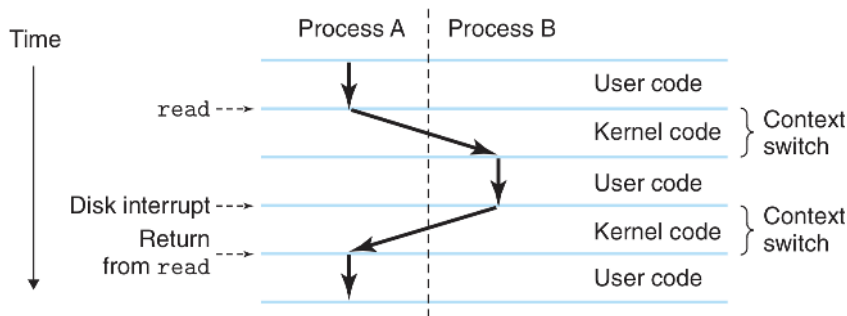


- It is no longer the compiler and the hardware that finds and exploits parallelism for the programmer.
- The programmer is responsible of designing algorithms that expose parallelism, and to exploit that parallelism using multiple cores.

# Operating System Supported Multiprocessing

A UNIX *process* is the Operating System's abstraction for a running program.

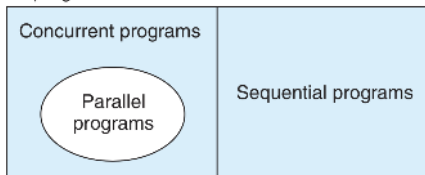
- Multiprocessing is the ability of the OS to run more than one process *concurrently*.
- Concurrency means that processes make progress on different tasks in the same time window.
- Concurrent processes can run different tasks using time slicing, that is the instructions of one process are interleaved with the instructions of another process. Does this require multiple cores?



# Operating System Supported Multiprocessing

- Concurrent processes can also run in *parallel*: they perform different tasks *simultaneously*.
- Parallelism refers to the use of concurrency to make a system/program run faster.

All programs



- The OS is ultimately responsible to map processes efficiently to the hardware, e.g. map different process to different cores, if available.
- Each process is assigned a process identifier (PID) that is unique across the entire system (try using the “ps” program).
- Processes usually correspond to completely different programs with their own set of instructions, global data, stack and heap (different virtual address space).
- UNIX processes can, however, communicate using `pipe()`, `socket()` and various OS-supported shared-memory areas.

# Operating System Supported Multiprocessing

Any UNIX process can spawn new processes (called “children”) using the UNIX `fork()` utility.

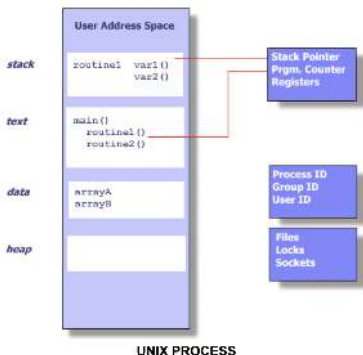
- `fork()` creates a clone of the parent program replicating the code, global variables and stack.
- In order to use `fork()` you have to `#include <unistd.h>`
- The only difference between the parent and the child is that the latter returns from `fork()` with 0, while the parent returns with the PID of the child
- The execution of the parent/child resumes from the `fork()` call

```
#include<stdio.h>
#include<unistd.h>

int main()
{
    fork();
    printf("This will be printed two times!\n");
    return 0;
}
```

# Operating System Supported Multiprocessing

It is possible to use basic UNIX utilities like `fork()`, `socket()` and `pipe()` and shared memory segments to generate a parallel program using multiple cores



- `fork()` is a “heavy weight” operation taking substantial time, e.g. it may have to replicate a process that has 1GB of memory.
- `fork()` is, however, used in a number of message passing protocols whereby multiple copies of a program are created once at the beginning of execution and then communicate via a series of messages.
- Message passing makes it complicated and potentially inefficient for flows to communicate, especially if they share large amounts of data.

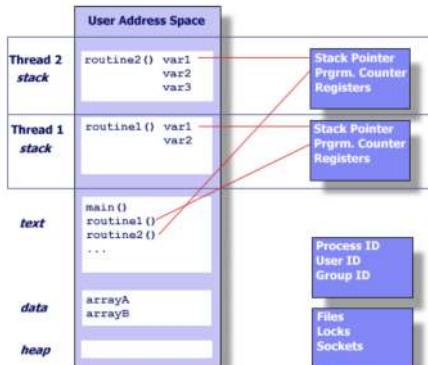
How can we avoid the flaws of process-based concurrency?

# Parallel Programming with Threads

A *thread* is a logical flow that runs within the context of a process.

- So far we discussed programs that consisted of a single thread per process.
- Multiple “independent” threads can be added to an existing process rather than starting a new process.
- Threads are able to be scheduled by the OS and run as independent entities largely because they duplicate only the bare essential resources that enable them to exist as executable code.

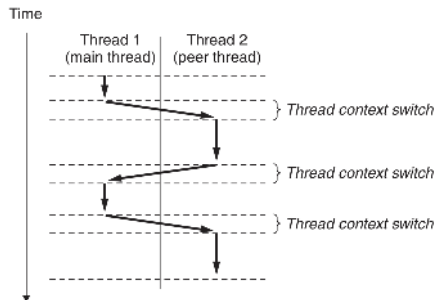
# Parallel Programming with Threads



THREADS WITHIN A UNIX PROCESS

- Each thread has its own *thread context*: a unique integer thread ID (TID), stack, stack pointer, program counter, general-purpose registers, and condition codes.
- However, all threads running in a process share the entire virtual address space of that process, including its code, data, heap, shared libraries, and open files.
- Spawning a thread is a “light-weight” (but absolutely not free!!) operation because most of the overhead has already been incurred through the creation of its process.

# Thread Execution Model



- Each process is created with a single thread called the *main* thread.
- The main thread can create a *peer* thread, and the two run concurrently.
- If they are scheduled on the same CPU/core, the OS at some point will pass control from the main to the peer thread via a context switch.
- If they are scheduled by the OS on different cores they run in parallel.
- Threads associated with a process form a pool of peers, where each one can kill or wait for any other to terminate.

# POSIX Threads (Pthreads)

Historically, hardware vendors have implemented their own proprietary versions of threads, making it difficult for programmers to develop portable threaded applications.

- POSIX Threads provide a standard interface, specified by the IEEE 1003.1 (1995) standard, for manipulating threads from C programs.
- Pthreads are defined as a set of C language programming types and procedure calls, implemented with an `#include <pthread.h>` header file and a thread library (this may be part of another library, such as `libc`, in some implementations).
- The specification contains about 60 functions that allow programs to create, kill, and reap threads, to share data safely with peer threads, and to notify peers about changes in the system state.

| Compiler / Platform     | Compiler Command            | Description |
|-------------------------|-----------------------------|-------------|
| INTEL<br>Linux          | <code>icc -pthread</code>   | C           |
|                         | <code>icpc -pthread</code>  | C++         |
| PGI<br>Linux            | <code>pgcc -lpthread</code> | C           |
|                         | <code>pgCC -lpthread</code> | C++         |
| GNU<br>Linux, Blue Gene | <code>gcc -pthread</code>   | GNU C       |
|                         | <code>g++ -pthread</code>   | GNU C++     |

# POSIX Threads (Pthreads)

Let's look at an example

```
1  #include <pthread.h>
2  void *thread(void *vargp);
3
4  int main()
5  {
6      pthread_t tid;
7      Pthread_create(&tid, NULL, thread, NULL);
8      Pthread_join(tid, NULL);
9      exit(0);
10 }
11
12 void *thread(void *vargp) /* Thread routine */
13 {
14     printf("Hello, world!\n");
15     return NULL;
16 }
```

# POSIX Threads (Pthreads)

## Creating threads

```
#include <pthread.h>
typedef void *(func)(void *);

int pthread_create(pthread_t *tid, pthread_attr_t *attr,
                  func *f, void *arg);
```

Returns: 0 if OK, nonzero on error

- Creates a new thread and runs the thread routine `f` in the context of the new thread and with an input argument of `arg`.
- The thread routine `f` takes as input a single generic pointer and returns a generic pointer. If you want to pass multiple arguments to a thread routine, then you should put the arguments into a structure and pass a pointer to the structure. Similarly, if you want the thread routine to return multiple arguments, you can return a pointer to a structure.
- When `pthread_create` returns, argument `tid` contains the ID of the newly created thread, which can be determined also using

# POSIX Threads (Pthreads)

## Terminating threads

- Implicit termination when its top-level thread routine returns.
- Explicit termination using `pthread_exit`. Explicit termination of main thread causes to wait all other peers to terminate.

```
#include <pthread.h>
```

```
void pthread_exit(void *thread_return);
```

Returns: 0 if OK, nonzero on error

- A peer thread calls `exit` terminating the process and all its threads.

```
#include <pthread.h>
```

```
int pthread_cancel(pthread_t tid);
```

Returns: 0 if OK, nonzero on error

- A peer thread calls `pthread_cancel` with the ID of the current thread.

# POSIX Threads (Pthreads)

## Returning and reaping

```
#include <pthread.h>
```

```
int pthread_join(pthread_t tid, void **thread_return);
```

Returns: 0 if OK, nonzero on error

- Current thread waits for thread `tid` to terminate, blocking until it does so.
- Assigns the generic (void \*) pointer returned by the thread routine to the location pointed to by `thread_return`.
- Reaps any memory resources (e.g. stack) held by the terminated thread.

# POSIX Threads (Pthreads)

## Detaching threads

- A thread is either joinable or detachable.
- Joinable: its memory resources (such as the stack) are not freed until it is reaped by another thread.
- A detached thread cannot be reaped by other threads. Its memory resources are freed automatically by the system when it terminates.
- To avoid memory leaks, each joinable thread should either be explicitly reaped by another thread, or detached by a call to `pthread_detach`

```
#include <pthread.h>

int pthread_detach(pthread_t tid);
```

Returns: 0 if OK, nonzero on error

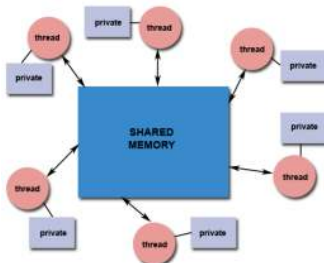
- Threads can detach themselves by calling `pthread_detach` with an argument of `pthread_self`.

# Pthreads Memory Model

The most attractive aspect of threads is the efficiency and easy with which they can share data

```
1 #include "csapp.h"
2 #define N 2
3 void *thresd(void *vargp)
4 {
5     char **ptr; /* Global variable */
6
7     int main()
8     {
9         int i;
10         pthread_t tid;
11         char *args[N] = {
12             "Hello from dad",
13             "Hello from bar"
14         };
15
16         ptr = args;
17         for (i = 0; i < N; i++)
18             Pthread_create(&tid, NULL, thresd, (void *)i);
19         Pthread_exit(NULL);
20     }
21
22 void *thresd(void *vargp)
23 {
24     int myid = (int)vargp;
25     static int cnt = 0;
26     printf("[X%d]: %s (cnt=%d)\n", myid, ptr[myid], ++cnt);
27     return NULL;
28 }
```

- The price to pay is the potential for complicated bugs that do not appear in serial code.
- To write correct threaded programs a clear understanding of the sharing mechanisms and risks is required

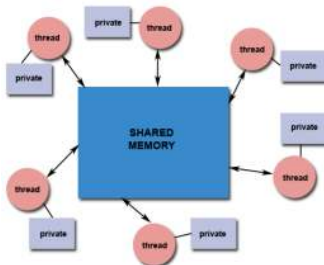


# Pthreads Memory Model

The most attractive aspect of threads is the efficiency and easy with which they can share data

```
1  #include "csapp.h"
2  #define N 2
3  void *thresd(void *vargp);
4
5  char **ptr; /* Global variable */
6
7  int main()
8  {
9      int i;
10     pthread_t tid;
11     char *args[N] = {
12         "Hello from foo",
13         "Hello from bar"
14     };
15
16     ptr = args;
17     for (i = 0; i < N; i++)
18         Pthread_create(&tid, NULL, thread, (void *)i);
19     Pthread_exit(NULL);
20 }
21
22 void *thresd(void *vargp)
23 {
24     int xpid = (int)vargp;
25     static int cnt = 0;
26     printf("[%d]: %s (cnt=%d)\n", xpid, ptr[xpid], ++cnt);
27     return NULL;
28 }
```

- Thread context: TID, stack, stack pointer, program counter, condition codes, registers values.
- Shared?

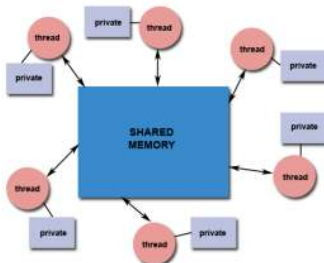


# Pthreads Memory Model

The most attractive aspect of threads is the efficiency and easy with which they can share data

```
1 #include "csapp.h"
2 #define N 2
3 void *thread(void *vargp)
4 {
5     char **ptr; /* Global variable */
6
7     int main()
8     {
9         int i;
10        pthread_t tid;
11        char *args[N] = {
12            "Hello from foo",
13            "Hello from bar"
14        };
15
16        ptr = args;
17        for (i = 0; i < N; i++)
18            Pthread_create(&tid, NULL, thread, (void *)i);
19        Pthread_exit(NULL);
20    }
21
22 void *thread(void *vargp)
23 {
24     int myid = (int)vargp;
25     static int cnt = 0;
26     printf("[X%d]: %s (cnt=%d)\n", myid, ptr[myid], ++cnt);
27     return NULL;
28 }
```

- Thread context: TID, stack, stack pointer, program counter, condition codes, registers values.
- Shared: process virtual address space → code, read/write data, heap, shared libraries, open files.

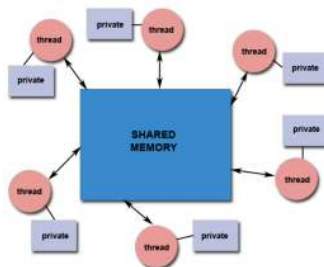


# Pthreads Memory Model

The most attractive aspect of threads is the efficiency and easy with which they can share data

```
1 #include "csapp.h"
2 #define N 2
3 void *thresd(void *vargp)
4 {
5     char **ptr; /* Global variable */
6
7     int main()
8     {
9         int i;
10         pthread_t tid;
11         char *args[N] = {
12             "Hello from foo",
13             "Hello from bar"
14         };
15
16         ptr = args;
17         for (i = 0; i < N; i++)
18             Pthread_create(&tid, NULL, thresd, (void *)i);
19         Pthread_exit(NULL);
20     }
21
22 void *thresd(void *vargp)
23 {
24     int myid = (int)vargp;
25     static int cnt = 0;
26     printf("[%d]: %s (cnt=%d)\n", myid, ptr[myid], ++cnt);
27     return NULL;
28 }
```

- Thread context  $\neq$  private! Why?

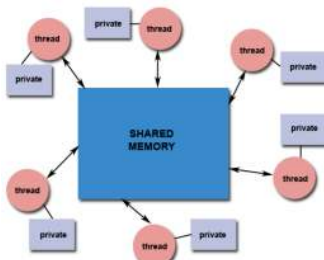


# Pthreads Memory Model

The most attractive aspect of threads is the efficiency and easy with which they can share data

```
1 #include <csapp.h>
2 #define N 2
3 void *thread(void *argp);
4 char **ptr; /* Global variable */
5
6 int main()
7 {
8     int i;
9     pthread_t tid;
10    char *args[N] = {
11        "Hello from foo",
12        "Hello from bar"
13    };
14
15    ptr = args;
16    for (i = 0; i < N; i++)
17        Pthread_create(&tid, NULL, thread, (void *)i);
18    Pthread_exit(NULL);
19 }
20
21 void *thread(void *argp)
22 {
23     int ayid = (int)argp;
24     static int cnt = 0;
25     printf("[M]: %s (cnt=%d)\n", ptr[ayid], ++cnt);
26     return NULL;
27 }
```

- Registers, CC, PC, SP are private. Thread stack is usually accessed independently by each thread. However, thread stacks are not protected!
- If a thread accesses somehow a pointer to another thread's stack, it can read and write to it!



# Pthreads Memory Model

```
1 #include <csapp.h>
2 #define N 2
3 void *thread(void *argp)
4 {
5     char **ptr; /* Global variable */
6
7     int main()
8     {
9         int i;
10        pthread_t tid;
11        char *args[N] = {
12            "Hello from foo",
13            "Hello from bar"
14        };
15
16        ptr = args;
17        for (i = 0; i < N; i++)
18            Pthread_create(&tid, NULL, thread, (void *)i);
19        Pthread_exit(NULL);
20    }
21
22 void *thread(void *argp)
23 {
24     int myid = (int)argp;
25     static int cnt = 0;
26     printf("[%d]: %s (cnt=%d)\n", myid, ptr[myid], ++cnt);
27     return NULL;
28 }
```

- global variables: declared outside functions, only one copy in virtual memory area, readable/writable by any thread.
- local automatic variables: declared inside functions, each thread's stack contains its own instance of it. E.g. `tid` and `myid`.
- local static variables: declared inside functions with the static attribute. There is only one copy in virtual memory area, readable/writable by any thread.
- shared variables: a variable is shared only and only if one of its instances is referenced by more than one thread. E.g. `cnt`.

# Pthreads Summary

- Threads share a virtual address space within a single process which allows direct memory read/writes for communication
- The good: fast communication between threads
- The bad: the potential for complex bugs and additional complexity required for synchronisation
- Not all memory accesses are the same: NUMA, L3 cache
  - run `cpuinfo` on gadi and see the cache sharing information
  - run `numactl -H` on gadi and see the “distances”
- Good cache/memory use in threaded programs is more complex